

STRUCTURE

- A structure is a user-defined data type.
- Structure is nothing but group of data items of different type with a single name.

Syntax to Define Structure

- ▶ To define a structure, we need to use **struct** keyword.
- ▶ This keyword is reserved word in C language. We can only use it for structure and its object declaration.

Syntax

```
1 struct structure_name
2 {
3     member1_declaration;
4     member2_declaration;
5     . . .
6     memberN_declaration;
7 };
```

→ structure_name is name of custom type

→ memberN_declaration is individual member declaration

- ▶ Members can be normal variables, pointers, arrays or other structures.
- ▶ Member names within the particular structure must be distinct from one another.

Example to Define Structure

Example

```
1 struct student
2 {
3     char name[30]; // Student Name
4     int roll_no; // Student Roll No
5     float CPI; // Student CPI
6     int backlog; // Student Backlog
7 };
```

- ▶ You must terminate structure definition with **semicolon ;**.
- ▶ You **cannot assign value** to members inside the structure definition, it will cause **compilation error**.

Example

```
1 struct student
2 {
3     char name[30] = "ABC"; // Student Name
4     . . .
5 };
```

Create Structure variable

- ▶ A data type defines various properties about data stored in memory.
- ▶ To use any type we must declare its variable.
- ▶ Hence, let us learn how to create our custom structure type objects also known as **structure variable**.
- ▶ In C programming, there are two ways to declare a structure variable:
 1. Along with structure definition
 2. After structure definition

Create Structure Variable – Cont.

1. Declaration along with the structure definition

Syntax

```
1 struct structure_name
2 {
3     member1_declaration;
4     member2_declaration;
5     . . .
6     memberN_declaration;
7 } structure_variable;
```

Example

```
1 struct student
2 {
3     char name[30]; // Student Name
4     int roll_no; // Student Roll No
5     float CPI; // Student CPI
6     int backlog; // Student Backlog
7 } student1;
```

Create Structure Variable – Cont.

2. Declaration after Structure definition

Syntax

```
1 struct structure_name structure_variable;
```

Example

```
1 struct student
2 {
3     char name[30]; // Student Name
4     int roll_no; // Student Roll No
5     float CPI; // Student CPI
6     int backlog; // Student Backlog
7 };
8 struct student student1; // Declare structure variable
```

Access Structure member (data)

- ▶ Structure is a complex data type, we cannot assign any value directly to it using assignment operator.
- ▶ We must assign data to individual **structure members** separately.
- ▶ C supports two operators to access structure members, using a structure variable.
 1. Dot/period operator (.)
 2. Arrow operator (->)

Access Structure member (data) – Cont.

1. Dot/period operator (.)

- ↳ It is known as member access operator. We use **dot operator** to access members of simple structure variable.

Syntax

```
1 structure_variable.member_name;
```

Example

```
1 // Assign CPI of student1
2 student1.CPI = 7.46;
```

2. Arrow operator (->)

- ↳ In C language it is illegal to access a structure member from a pointer to structure variable using dot operator.
- ↳ We use **arrow operator** to access structure member from pointer to structure.

Syntax

```
1 pointer_to_structure->member_name;
```

Example

```
1 // Student1 is a pointer to student type
2 student1 -> CPI = 7.46;
```

Array of Structure

- ▶ It can be defined as the collection of multiple structure variables where each variable contains information about different entities.
- ▶ The array of structures in C are used to store information about **multiple entities of different data types**.

Syntax

```
1 struct structure_name
2 {
3     member1_declaration;
4     member2_declaration;
5     ...
6     memberN_declaration;
7 } structure_variable[size];
```

Nested Structure

- ▶ When a **structure contains another structure**, it is called **nested structure**.
- ▶ For example, we have two structures named **Address** and **Student**. To make Address nested to Student, we have to define Address structure before and outside Student structure and create an object of Address structure inside Student structure.

Syntax

```
1 struct structure_name1
2 {
3     member1_declaration;
4     member2_declaration;
5     ...
6     memberN_declaration;
7 };
8 struct structure_name2
9 {
10    member1_declaration;
11    member2_declaration;
12    ...
13    struct structure1 obj;
14 };
```

- **Arrays Within Structures (Structure containing an Array)**

- A **structure** can contain many data types, including **arrays**.
- This is useful when you want to group related information.

```
#include <stdio.h>
struct Student {
    char name[20];
    int marks[5];    // Array inside structure
};

int main() {
    struct Student s = {"Riya", {78, 85, 92, 88, 91}};
    printf("Name: %s\n", s.name);
    for(int i = 0; i < 5; i++) {
        printf("Mark %d = %d\n", i+1, s.marks[i]);
    }
    return 0;
}
```

What is Union?

- ▶ Union is a **user defined data type** similar like Structure.
- ▶ It holds different data types in the **same memory location**.
- ▶ You can define a **union** with various members, but only one member can hold a value at any given time.
- ▶ Union provide an efficient way of using the same memory location for multiple-purpose.

Example to Define Union

Example

```
1 union student
2 {
3     char name[30]; // Student Name
4     int roll_no; // Student Roll No
5     float CPI; // Student CPI
6     int backlog; // Student Backlog
7 } student1;
```

- ▶ You must terminate union definition with **semicolon ;**.
- ▶ You cannot assign value to members inside the union definition, it will cause compilation error.

Example

```
1 union student
2 {
3     char name[30] = "ABC"; // Student Name
4     . . .
5 } student1;
```

Structure Vs. Union

COMPARISON	STRUCTURE	UNION
Basic	The separate memory location is allotted to each member of the structure.	All members of the 'union' share the same memory location.
keyword	'struct'	'union'
Size	Size of Structure = sum of size of all the data members.	Size of Union = size of the largest member.
Store Value	Stores distinct values for all the members.	Stores same value for all the members.
At a Time	A structure stores multiple values, of the different members, of the structure.	A union stores a single value at a time for all members.
Declaration	<pre>struct ss { int a; float f; char c };</pre> 1 byte for c 2 bytes for a 4 bytes for f	<pre>union uu { int a; float f; char c };</pre> 4 bytes f a c